

JCMS — Phase-by-Phase Build Prompts for Antigravity

How to use this file:

- Feed the prompts to Antigravity **in order**, one phase at a time, in the same workspace/repo.
 - Each phase prompt is self-contained: paste it as-is. Don't skip ahead — later phases assume earlier ones are committed.
 - Each phase ends with an explicit instruction to commit and push, so you get a clean, reviewable git history instead of one giant commit.
 - Attach `PROJECT_CONTEXT.md` (the brief you already have) to the very first prompt so Antigravity has full context before touching anything.
 - Wherever a phase says "Dev A" or "Dev B", that's just which vertical the code belongs to for your portfolio narrative — Antigravity builds all of it, you're just tracking ownership for your README/resume story.
-

Phase 0 — Repo Scaffolding & Environment

You are setting up the foundation for a project called JCMS (Judicial Case Management System), a full-stack, AI-powered court operations platform. I've attached PROJECT_CONTEXT.md with the full architecture, schema, API contracts, and feature spec – read it fully before doing anything.

Do the following:

1. Initialize a git repo (if not already) and create this monorepo structure:
 - apps/api (NestJS + TypeScript backend)
 - apps/web (React + Vite + TypeScript + TailwindCSS + shadcn/ui frontend)
 - packages/shared-types (shared TS types/interfaces used by both api and web)
 - infra/ (docker-compose.yml, later Terraform/k8s manifests)
 - .github/workflows/ (empty for now, CI comes in a later phase)
 - CONTRACTS.md (empty placeholder, to be filled in Phase 1)
 - README.md (project overview, tech stack table, how to run locally)
2. Set up a package manager workspace (npm/pnpm workspaces) so apps/api, apps/web, and packages/shared-types can reference each other.
3. Scaffold apps/api as a fresh NestJS project (TypeScript, strict mode).
4. Scaffold apps/web as a fresh Vite + React + TypeScript project. Install and configure TailwindCSS and shadcn/ui. Install TanStack Query and Zustand.
5. Create infra/docker-compose.yml with three services for local dev:
 - postgres (with the pgvector extension image, e.g. ankane/pgvector or pgvector/pgvector)
 - redis
 - minio (S3-compatible object storage, with a default bucket auto-created)
6. Add a root .env.example covering DB URL, Redis URL, JWT secrets, MinIO credentials, and placeholders for AI provider API keys (OpenAI/Gemini) – do NOT put real secrets anywhere.
7. Write the README with: project description, architecture diagram (as a mermaid diagram in markdown), tech stack table, and "getting started" instructions (docker compose up, install deps, run migrations, start dev servers).

Do not implement any business logic yet – this phase is pure scaffolding.

When done, run the projects to confirm they boot with no errors, then:

```
git add -A
```

```
git commit -m "chore: scaffold monorepo (api, web, shared-types, docker-  
compose)"
```

```
git push
```

Phase 1 — Database Schema, Contracts & Auth (Dev B foundation)

Continuing the JCMS project (see PROJECT_CONTEXT.md and the repo scaffolded in Phase 0).

Goal: implement the core database schema, the shared contracts doc, and authentication.

1. In apps/api, set up Prisma with PostgreSQL. Implement this schema as Prisma models

(translate types/enums faithfully, use UUIDs for primary keys, add appropriate indexes and relations):

User, Court, Courtroom, Judge, Case, Party, Hearing, Document, PriorityFactor, AuditLog, Notification

Use exactly the fields and enums described in PROJECT_CONTEXT.md section 4.1.

Add the pgvector extension and a `document\_chunk` table per section 4.2, with an ivfflat cosine-similarity index on the embedding column.

2. Write and run the initial Prisma migration. Add a `prisma/seed.ts` that seeds:

- 1 Court with 2 Courtrooms
- 4 Users, one per role (Judge, Advocate, Registrar, Admin) with hashed passwords
- 1 sample Case with a Party and a Hearing

3. Implement JWT authentication in NestJS:

- POST /api/auth/login (email + password -> access + refresh token)
- POST /api/auth/refresh
- POST /api/auth/logout
- Password hashing with argon2 (or bcrypt if argon2 isn't available in the environment)
- A `@Roles()` decorator + RolesGuard that restricts routes by User.role (Judge | Advocate | Registrar | Admin | Public)

4. Create CONTRACTS.md at the repo root and document these 5 integration contracts in full,

verbatim from PROJECT_CONTEXT.md section 4.4 and surrounding text:

- Shared TypeScript types package contents
- AIPrioritySuggestion interface (exact shape given in the brief)

- Auth guard / @Roles() decorator shape
- WebSocket event payload shapes (queue:updated, hearing:scheduled, emergency:alert)
- DocumentSummary type for case-document linking

5. Put the AIPrioritySuggestion interface and any other cross-cutting types into

packages/shared-types so both apps/api and apps/web can import them.

6. Write basic e2e tests (supertest or NestJS's built-in e2e setup) for the three auth

endpoints, covering: successful login, wrong password, expired/invalid refresh token,
and a role-guarded route rejecting the wrong role.

Run the migration, run the seed script, run the tests, confirm everything passes, then:

```
git add -A
```

```
git commit -m "feat: db schema, prisma migration+seed, JWT auth, RBAC guard, CONTRACTS.md"
```

```
git push
```

Phase 2 — Case Lifecycle & Priority Scheduling Engine (Dev A core)

Continuing JCMS. Auth, schema, and contracts are in place from Phase 1.

Goal: implement the case lifecycle backend and the weighted priority scheduling engine.

1. Implement full CRUD + status-transition logic for Case:

- POST /api/cases
- GET /api/cases/:id
- GET /api/cases?status=&court=&q= (filterable, paginated)
- PATCH /api/cases/:id/status
- POST /api/cases/:id/emergency-flag
- GET /api/cases/:id/audit-trail

Enforce the status state machine exactly as specified:

Filed -> Verification -> Listed -> Heard -> Disposed (with a Returned branch from

Verification). Reject any transition that isn't a valid edge in this state machine,

with a clear error message naming the invalid transition.

2. Implement the Digital Audit Log:

- Every case status change, emergency flag, and priority score change writes an

AuditLog row with before_state/after_state JSONB snapshots.

- Each AuditLog entry's `hash` is computed over (entity_id, action, after_state, prev_hash)

so the chain is tamper-evident. `prev\_hash` links to the previous entry for that entity.

- Implement GET /api/cases/:id/audit-trail returning the full chain.

- Implement a verification endpoint/service method that walks the chain and confirms

no hash has been tampered with; write a test that mutates a row directly in the DB

and confirms verification correctly fails.

3. Implement the Weighted Priority Scheduling Engine:

- PriorityFactor types: Age, StatutoryUrgency, EmergencyFlag, VulnerabilityFlag, each

with a configurable weight.

- A deterministic scoring function that combines these factors into Case.priority_score.

- Back the live queue with a Redis sorted set per court (`queue:court:{courtId}`), score

```
    = priority_score, member = case_id, so reordering is O(log N).
- POST /api/scheduler/recompute recalculates and re-syncs the Redis sorted
set from Postgres.
- GET /api/courts/:courtId/queue reads the live ordered queue from Redis
(falling back to
  Postgres if Redis is cold).
- POST /api/courts/:courtId/queue/reorder allows a manual registrar
override, which must
  also write an AuditLog entry with a `reason` field.
- Write unit tests for the scoring function covering: a normal case, an
emergency-flagged
  case that should jump the queue, and a tie-break rule you document in a
code comment.
```

4. Implement the Courtroom Load Balancer:

```
- POST /api/hearings, PATCH /api/hearings/:id, GET
/api/courtrooms/:id/availability
- Conflict detection: reject a hearing creation/update that double-books a
courtroom or
  a judge for an overlapping time slot, with a descriptive 409 response.
```

Run all tests, confirm green, then:

```
git add -A
git commit -m "feat: case lifecycle state machine, hash-chained audit log,
priority scheduling engine, courtroom allocation"
git push
```

Phase 3 — Registrar Emergency Workflow (Dev A)

Continuing JCMS.

Goal: implement the emergency-petition override workflow described in PROJECT_CONTEXT.md section 3 (Dev A – Registrar Portal).

1. Implement the emergency case state machine as its own tracked sub-flow, distinct from the main case status:
Filed(Emergency) -> Registrar Triage -> Judge Acceptance -> Slot Injection -> Notification fan-out

Each transition should be its own service method, each writing an AuditLog entry, and each guarded so only the correct role can perform it (Registrar triages, Judge accepts).

2. "Slot Injection" must call into the Courtroom Load Balancer from Phase 2 to find the earliest non-conflicting slot, inserting the case at the top of the relevant Redis sorted-set queue.

3. "Notification fan-out" should create Notification rows for all relevant parties (advocate, assigned judge, registrar) – implement this as a simple synchronous write for now; note in a code comment that this is the natural place to later hook in the BullMQ queue and email/SMS channels once those are built.

4. Expose whatever new endpoints this workflow needs (e.g. POST /api/cases/:id/emergency-flag triggers entry into this flow; add POST /api/cases/:id/emergency/triage and POST /api/cases/:id/emergency/accept).

5. Write integration tests covering: happy path end-to-end through all five states, a registrar attempting to skip a state (should fail), and a non-registrar/non-judge attempting a restricted transition (should be 403).

Run tests, confirm green, then:

```
git add -A
git commit -m "feat: emergency petition override workflow (triage -> judge
acceptance -> slot injection -> notification)"
git push
```

Phase 4 — Document Vault & Public Portal (Dev B)

Continuing JCMS.

Goal: document storage and the public-facing portal.

1. Implement the Document Vault:

- POST /api/cases/:id/documents (multipart upload) storing files in MinIO (S3-compatible),
versioned by filename+incrementing version number, with a SHA-256 checksum computed
and stored on the Document row.
- GET /api/documents/:id/download (signed URL or streamed response).
- A virus-scan job stub using BullMQ: enqueue a job on upload, a worker processes it
(use a stub/mock scanner for now – e.g. always returns "clean" – but structure the
job so a real ClamAV integration could be swapped in later), and updates Document.virus_scan_status when done.

2. Implement the Public/Litigant Portal endpoints:

- GET /api/public/cause-list?court=&date= – public daily cause list, no auth required,
but must NOT leak any field beyond what's appropriate for public consumption (no
internal notes, no full party contact info).
- GET /api/public/case-status?caseNumber= – public case status lookup by case number only.
- Use Postgres full-text search (to_tsvector/to_tsquery) for case-number / party-name
lookup, since the brief specifies Postgres FTS for MVP (Elasticsearch is a documented
future phase, don't build it now).

3. Write tests: upload -> download round-trip with checksum verification, a rejected

upload of an oversized/wrong-mimetype file, and public endpoints confirmed to omit
sensitive fields.

Run tests, confirm green, then:

```
git add -A
```

```
git commit -m "feat: document vault (MinIO storage, versioning, checksum,
```

```
virus-scan job stub), public cause-list and case-status lookup"
git push
```

Phase 5 — Real-Time WebSocket Layer (Dev B)

Continuing JCMS.

Goal: wire up real-time updates across the platform per PROJECT_CONTEXT.md section 5.5.

1. Set up a Socket.IO gateway in NestJS with namespace/room structure `court:{courtId}`.
2. Implement and emit these events with the exact payload shapes documented in CONTRACTS.md:
 - queue:updated – fired whenever the Phase 2 scheduling engine changes a court's Redis sorted-set order (manual reorder, recompute, or emergency slot injection)
 - hearing:scheduled – fired whenever a Hearing is created/updated (Phase 2)
 - emergency:alert – fired the moment a case enters "Filed(Emergency)" (Phase 3),
 - routed to the relevant judge's connected socket immediately
3. Add socket authentication: clients must present their JWT on connection; join `court:{courtId}` rooms only if their role/assignment permits it.
4. On the frontend (apps/web), add a thin WebSocket client hook (e.g. useCourtSocket) that connects, joins the right room based on the logged-in user's court, and exposes the three event types via a small pub/sub or React Query cache-patching pattern.
5. Write a test (can be a lightweight integration test spinning up the gateway) confirming that a queue reorder in Phase 2's service actually emits a queue:updated event with the documented payload shape.

Run tests, confirm green, then:

```
git add -A
git commit -m "feat: Socket.IO real-time layer (court rooms,
queue/hearing/emergency events) + frontend socket hook"
git push
```

Phase 6 — Frontend: Auth, Layout & Role Portals

Continuing JCMS. All backend APIs through Phase 5 exist and are working.

Goal: build the frontend shell and the four role-based portals, wired to the real API.

1. Set up TanStack Query for all API calls and Zustand for client-side auth/session state.

Implement login, token storage (memory + refresh flow, not localStorage for the access token), and route protection by role.

2. Build shared layout: top nav, role-aware sidebar, and a design system pass using shadcn/ui components consistent with a clean, professional "govtech/legal" look.

3. Build the four portals as separate route groups, each pulling from the real endpoints

built in earlier phases (no mock data):

- Judge Dashboard: docket view (live via WebSocket from Phase 5), disposition recording

form, courtroom management view, bench notes field on a case.

- Registrar Portal: filing verification queue, the emergency-triage console (Phase 3

workflow, with clear visual state of where each emergency case sits in its flow),

manual queue reorder UI (drag-and-drop calling the reorder endpoint).

- Advocate Portal: e-filing form (creates a Case via POST /api/cases), case tracking list,

hearing calendar view.

- Public Portal: no-login cause list search and case-status lookup, using the Phase 4

public endpoints. This is the only portal accessible without auth.

4. Add loading/error/empty states for every data view – do not leave any screen able to

silently show nothing on a failed fetch.

5. Add a handful of component tests (React Testing Library) for the riskiest interactions:

emergency-triage console state transitions, and the reorder drag-and-drop calling the

right endpoint with the right payload.

Run the app locally against the dev docker-compose stack, confirm each portal works

end-to-end against real data, then:

```
git add -A
```

```
git commit -m "feat: frontend shell, auth flow, and four role-based portals  
wired to live API + WebSocket"
```

```
git push
```

Phase 7 — AI/LLM Feature Pipeline (Dev B)

Continuing JCMS. This phase implements PROJECT_CONTEXT.md section 5 in full. All AI features must be provider-agnostic where reasonable (OpenAI/Gemini/local via a thin adapter interface) and must never auto-commit anything without human review, per the brief's constraint.

1. Petition Categorization & Extraction:

- POST /api/ai/petitions/:id/extract
- OCR fallback (Tesseract) for scanned PDFs; direct text extraction for digital PDFs.
- LLM call with a JSON-schema-constrained structured output producing: case title, party names, case type, key dates, relief sought.
- Deterministic regex/fuzzy-match validation pass before anything touches the DB.
- The result is returned to the Registrar Portal as a pre-filled, editable form – do NOT auto-commit to the Case table. The registrar's explicit "confirm" action is what actually writes the row.

2. AI Priority Urgency Classifier:

- POST /api/ai/cases/:id/priority-suggestion
- Returns exactly the AIPrioritySuggestion shape from CONTRACTS.md (urgency_class, rationale, flagged_factors, suggested_weight_delta bounded -15..+15).
- This suggestion feeds into Phase 2's scoring engine as a bounded delta only – the deterministic engine remains the source of truth for the final score. Log every suggestion alongside the human-adjusted final score for future eval.
- Implement a cheap-model-first, escalate-on-ambiguity cost strategy (e.g. a smaller/cheaper model classifies first; only calls the larger model if confidence is low or the case has conflicting signals).

3. RAG-based Case Summarization:

- POST /api/ai/cases/:id/summarize
- Chunk all of a case's documents (~500-1000 tokens, with overlap), store chunks + embeddings in the document_chunk/pgvector table from Phase 1.

- Retrieval: filter by case_id first, then rank by cosine similarity (two-stage).
 - Generation: produce a structured summary (Parties, Timeline, Key Evidence, Prior Orders, Open Issues), with each claim citing the source chunk/page it came from.
4. AI Stenographer:
- POST /api/ai/hearings/:id/draft-order
 - Speech-to-text (Whisper API or self-hosted faster-whisper) + diarization on an uploaded/recorded hearing audio file.
 - LLM structures the diarized transcript into a draft order (case number, appearances, submissions, directions, next date).
 - The output must be explicitly labeled "AI-generated draft – pending judicial review" in both the API response and anywhere it's rendered in the UI, and can only become the Hearing.order_summary after the judge explicitly approves/edits it.
5. Add an eval harness: a small labeled test set (you can synthesize ~10-15 example petitions/transcripts) to sanity-check extraction accuracy, plus prompt-version tracking (store which prompt version produced which stored suggestion, for future comparison).
6. Wire minimal frontend UI for each of these four features into the relevant portal from Phase 6 (extraction into Registrar's filing verification, priority-suggestion as an assistive hint in the queue view, summarization as a "Summarize File" button on a case, draft-order into the Judge Dashboard) – each clearly marked as AI-assisted, human-reviewed.

Run whatever tests are feasible without live API keys (mock the LLM/OCR/STT calls in tests), confirm the app still boots and non-AI flows are unaffected, then:

```
git add -A
git commit -m "feat: AI pipeline (petition extraction, priority-suggestion classifier, RAG summarization, AI stenographer) with human-in-the-loop gates"
git push
```

Phase 8 — CI/CD, Containerization & Observability

Continuing JCMS. This is the "production/cloud-readiness" phase.

1. Write production Dockerfiles for apps/api and apps/web (multi-stage builds, small final images).
2. Write a GitHub Actions workflow (.github/workflows/ci.yml) that on every push/PR:
 - installs deps, runs lint, runs unit + e2e tests for apps/api and apps/web
 - builds both Docker images
 - (optionally) pushes images to GHCR on merge to main
3. Add Prometheus metrics exposure from the NestJS app (basic request/latency/error-rate metrics) and a minimal Grafana dashboard JSON checked into infra/observability/.
4. Add Sentry (or a stub if no DSN is available) for error tracking on both frontend and backend, gated behind an env var so it's a no-op in local dev without a DSN.
5. Write a first-pass Terraform module skeleton (infra/terraform/) for the target cloud deployment (ECS or a small Kubernetes setup) – this can be a well-commented skeleton rather than something you actually apply, since there's no live cloud account yet.
6. Update the README with: CI badge, how to run the full test suite, how to build/run the Docker images locally, and a short "Deployment" section pointing at the Terraform skeleton.

Confirm the CI workflow is syntactically valid and the Docker images build locally, then:

```
git add -A
git commit -m "chore: CI pipeline, production Dockerfiles, Prometheus/Grafana + Sentry observability, Terraform skeleton"
git push
```

Phase 9 — Hardening, Load Testing & Demo Polish

Continuing JCMS. This is the final portfolio-polish phase.

1. Run a k6 load test script (checked into infra/load-tests/) against the priority queue and case-listing endpoints; document baseline results (requests/sec, p95 latency) in README or a PERFORMANCE.md.
2. Do a security hardening pass: rate limiting on auth and public endpoints, input validation/sanitization audit across all controllers, helmet/CORS config review, dependency vulnerability scan (npm audit or similar) with any high/critical issues fixed.
3. Seed a richer demo dataset (multiple courts, several cases across every status and both emergency and non-emergency, a few documents, some AI-generated summaries) so the app looks alive out of the box for recruiters/interviewers.
4. Write a DEMO_SCRIPT.md walking through a 5-minute demo path touching every major feature (login as each role, file a case, trigger the emergency workflow, watch the live queue update via WebSocket, generate an AI summary, view the tamper-evident audit trail).
5. Final README pass: architecture diagram, feature list, tech stack, "why these choices" section for portfolio/interview talking points, and clear local setup instructions.

Confirm the seeded demo works end-to-end following DEMO_SCRIPT.md, then:

```
git add -A
git commit -m "chore: load testing baseline, security hardening pass, rich demo seed data, demo script, final README"
git push
```

Notes on using these prompts

- If Antigravity loses context between phases (new session), re-attach `PROJECT_CONTEXT.md` plus a short note: "We're resuming JCMS — Phases 0–N are already committed to this repo, see git log / README for what exists. Continue with Phase N+1 below." then paste that phase's prompt.

- You (or your teammate) finalized NestJS over Spring Boot as the default in the brief — if you actually pick Spring Boot instead, swap "NestJS" for "Spring Boot + JPA/Hibernate" in Phase 0 and Phase 1's prompts before pasting them; everything else stays the same.
- Each phase is scoped to be independently reviewable in a PR — feel free to have AntigraVity open a PR per phase instead of committing straight to main if you want review checkpoints.